Website themes

A theme defines how your website looks: layout, navigation, colors, typography, and output-specific styling. Every *Calepin* site uses one. The default theme works with zero configuration. When you want more customization, you can copy (or "eject") any built-in theme into your project and edit it like the rest of your source files.

Choosing

Set `theme` in your site's `calepin.toml`:

```
theme = "calepin"           # the default documentation theme
theme = "academic"          # a built-in academic site theme
theme = "themes/my-theme"   # a local theme directory
theme = false               # no theme: raw, unstyled output
```

The same values work with `--theme` on `calepin compile` and inside a document with `#calepin.setup(theme: ...)`. When several are set during a compile, the command line wins, then the document, then `calepin.toml`. `calepin watch` does not have a `--theme` option; it uses the document setting when present, otherwise the website's `calepin.toml` setting, otherwise the default theme.

Built-in

Calepin ships with two built-in themes:

- **calepin**: the default documentation site layout, with sidebar navigation, a top bar, previous and next page links, a table of contents, dark mode, copy buttons on code blocks, and rendered/source/PDF view switching.
- **academic**: a personal homepage layout with top navigation instead of a sidebar, designed for profile pages, teaching materials, publication lists, projects, talks, and posts. Run `calepin new academic` to scaffold a complete starter site built on it.

Local directory

A local theme is a directory in your project. Point `theme` at that directory:

```
theme = "themes/my-theme"
```

Use a local directory when you want project-specific HTML, CSS, or JavaScript. Missing files fall back to the built-in `calepin` theme, so a small local theme can override just one file.

Customizing

Built-in themes are compiled into the *Calepin* binary, so you cannot edit them directly. Instead, copy one into your project and edit the copy:

```
calepin new theme                     # copies the default theme to themes/calepin/
calepin new theme --theme academic    # copies the academic theme to themes/academic/
calepin new theme themes/my-theme --theme academic
```

Then point your site at the copy:

```
theme = "themes/calepin"
```

The copy is yours: edit its HTML, CSS, and JavaScript files freely, and check them into version control. *Calepin* upgrades will never touch them.

Structure and files

A theme can provide HTML templates for website pages and single documents:

```
themes/my-theme/
  site.html         # layout for website pages
  document.html     # layout for a single document rendered to HTML
  partials/         # reusable template fragments
  styles/           # CSS files, loaded in filename order
  scripts/          # JavaScript files, loaded in filename order
```

`site.html` and `document.html` use the MiniJinja template language, which follows familiar Jinja2 syntax. On website builds, templates receive a `site` object describing the site and current page.

From the `calepin.toml`, the template receives:

- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.favicon`

These variables are automatically computed and made available for navigation:

- `site.sidebar`, `site.sidebar_sections`
- `site.navbar_left`, `site.navbar_center`, `site.navbar_right`
- `site.toc`
- `site.page_title`
- `site.current_url`
- `site.language`, `site.languages`, `site.translations`

Here is a small but complete `site.html`:

```
{{ doc.head }}
  {% if site.page_title or site.title %}
  <title>{% if site.page_title %}{{ site.page_title }}{% if site.title %} |
{{ site.title }}{% endif %}{% else %}{{ site.title }}{% endif %}</title>
  {% endif %}
  {% if site.description %}
  <meta name="description" content="{{ site.description }}">
  {% endif %}
  {% if site.favicon %}
  <link rel="icon" href="{{ site.favicon }}">
  {% endif %}
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.
min.css">
  {% for style in styles %}
  <style>
{{ style.css }}
  </style>
  {% endfor %}
{{ doc.body_open }}
  {% include "partials/site-header.html" %}
  <div class="site-shell">
    {% include "partials/sidebar.html" %}
    <main class="site-main">
```

```
      {{ doc.body }}
    </main>
    {% if site.toc %}
    <aside class="site-toc">
      <strong>On this page</strong>
      <ul>
        {% for item in site.toc %}
        <li class="level-{{ item.level }}"><a href="{{ item.href }}">{{ item.label }}
</a></li>
        {% endfor %}
      </ul>
    </aside>
    {% endif %}
  </div>
  {% for script in scripts %}
  <script>
{{ script.content }}
  </script>
  {% endfor %}
{{ doc.body_close }}
```

`doc.head`, `doc.body_open`, and `doc.body_close` come from Typst's generated HTML. Keep them in the template unless you are deliberately replacing the whole document shell.

Partials

A partial is a reusable MiniJinja template fragment stored under `partials/`. Use partials for repeated HTML such as a header, footer, navigation list, search box, or analytics snippet.

Include a partial from `site.html`, `document.html`, or another partial:

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them, so a partial included by `site.html` can read `site.title`, `site.navbar_left`, `styles`, `scripts`, and the other website template variables.

For example, `partials/site-header.html` can render the brand and top navigation:

```
<header class="site-header">
  <nav>
    <ul>
      <li>
        <a href="{{ site.home_url }}">
          {% if site.logo %}
          <img src="{{ site.logo }}" alt="{{ site.logo_alt }}">
          {% elif site.title %}
          {{ site.title }}
          {% else %}
          Home
          {% endif %}
        </a>
      </li>
      {% for item in site.navbar_left %}
      {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
    <ul>
```

```
      {% for item in site.navbar_right %}
      {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
  </nav>
</header>
```

Then `partials/nav-item.html` can render one navigation link:

```
<li>
  <a href="{{ item.href }}" aria-label="{{ item.label }}"{% if item.active %} aria-
current="page"{% endif %}>
    {{ item.label_html }}
  </a>
</li>
```

`partials/sidebar.html` can use section state to render foldable navigation:

```
{% if site.sidebar_sections %}
<aside class="site-sidebar">
  {% for section in site.sidebar_sections %}
  {% if section.items %}
  <details{% if section.active %} open{% endif %}>
    <summary>{{ section.title }}</summary>
    <ul>
      {% for item in section.items %}
      <li><a href="{{ item.href }}"{% if item.active %} aria-current="page"{% endif
%}>{{ item.label_html }}</a></li>
      {% endfor %}
    </ul>
  </details>
  {% endif %}
  {% endfor %}
</aside>
{% endif %}
```

Shared files

When you run `calepin new theme`, *Calepin* writes the shared CSS and JavaScript as normal files in your theme. These are the pieces that the built-in themes use for typography, syntax highlighting, code output, dark mode, language selection, and copy buttons. They are copied into your theme so you can inspect, edit, delete, or replace them directly.

CSS

Shared CSS lives in `styles/`:

```
themes/calepin/
  styles/00-theme.css
  styles/01-code.css
  styles/02-widgets.css
```

The numeric prefixes matter only for load order. Theme CSS is loaded in filename order. Put broad variables and base rules first, then component rules, then project-specific overrides:

```
styles/
  00-theme.css
  01-code.css
  02-widgets.css
  90-overrides.css
```

JavaScript

Shared JavaScript lives in `scripts/`:

```
themes/calepin/
  scripts/00-theme-toggle.js
  scripts/01-language-picker.js
  scripts/02-copy-code.js
```

Theme JavaScript is loaded in filename order. Keep shared behavior before project-specific behavior:

```
scripts/
  00-theme-toggle.js
  01-language-picker.js
  02-copy-code.js
  90-custom.js
```

The built-in themes place their theme toggle, language picker, and output picker directly in `site.html`. The shared JavaScript files expect these attributes:

```
<button data-calepin-theme-toggle>Theme</button>
<select data-calepin-language-picker></select>
<select id="calepin-website-view-mode"></select>
```